

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Организация процессов и программирование в среде
Linux»
Тема: Управление потоками

Студент гр. 3311

Баймухамедов Р. Р.

Преподаватель

Лазарев Э. А.

Санкт-Петербург

2026

ЦЕЛЬ РАБОТЫ

Познакомиться с организацией потоков в Linux и способами синхронизации родительского процесса с созданными потоками.

ЗАДАНИЕ

1. Разработать программу, которая читает входной файл построчно и для каждой пары строк создаёт два параллельных потока. Один поток записывает нечётные строки в один выходной файл, другой - чётные строки в другой. Перед чтением следующей пары программа ожидает завершения обоих потоков.

2. Откомпилировать и запустить программу, затем вывести содержимое входного и выходных файлов.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Поток - это независимая последовательность выполнения внутри процесса. Потоки одного процесса разделяют адресное пространство, поэтому основной и созданные потоки могут использовать общие данные. При одновременной работе необходимо учитывать время жизни буферов и синхронизировать выполнение.

Функция `pthread_create` создаёт поток, `pthread_join` блокирует вызывающий поток до завершения указанного потока, а `pthread_exit` завершает текущий поток и позволяет передать статус завершения.

2 ПРОГРАММА

2.1 Подготовка входного и выходных файлов

Создадим исходный файл программы и входной файл с несколькими строками. Выходные файлы будут сформированы программой.

```
user@linuxlabs:~/lab04$ ls
input1.txt input2.txt program1.c program1_close program1_open program2.c
user@linuxlabs:~/lab04$ cat input2.txt
first string
second string
third string
fourth string
fifth string
sixth string
```

Рисунок 1 - Содержимое входного файла

2.2 Текст программы

Программа должна читать входной файл построчно, передавать нечётную и чётную строки двум разным потокам, ожидать оба потока с помощью `pthread_join` и затем переходить к следующей паре строк.

```
#define _POSIX_C_SOURCE 200809L

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct write_args {
    FILE *file;
    const char *line;
    int error;
};

static void *write_line(void *arg)
{
    struct write_args *args = arg;

    if (fputs(args->line, args->file) == EOF || fflush(args->file) == EOF) {
        args->error = 1;
    }
    return NULL;
}

static int start_thread(pthread_t *thread, struct write_args *args)
{
    int error = pthread_create(thread, NULL, write_line, args);
    if (error != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(error));
        return -1;
    }
    return 0;
}

static int wait_thread(pthread_t thread)
{
    int error = pthread_join(thread, NULL);
    if (error != 0) {
```

```

        fprintf(stderr, "pthread_join: %s\n", strerror(error));
        return -1;
    }
    return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr,
            "Использование: %s ВХОДНОЙ_ФАЙЛ НЕЧЁТНЫЕ_СТРОКИ "
            "ЧЁТНЫЕ_СТРОКИ\n",
            argv[0]);
        return EXIT_FAILURE;
    }

    FILE *input = fopen(argv[1], "r");
    FILE *odd = fopen(argv[2], "w");
    FILE *even = fopen(argv[3], "w");
    if (input == NULL || odd == NULL || even == NULL) {
        perror("fopen");
        if (input != NULL) fclose(input);
        if (odd != NULL) fclose(odd);
        if (even != NULL) fclose(even);
        return EXIT_FAILURE;
    }

    char *odd_line = NULL;
    char *even_line = NULL;
    size_t odd_capacity = 0;
    size_t even_capacity = 0;
    int result = EXIT_SUCCESS;

    while (getline(&odd_line, &odd_capacity, input) != -1) {
        ssize_t even_length = getline(&even_line, &even_capacity, input);
        struct write_args odd_args = { odd, odd_line, 0 };
        struct write_args even_args = { even, even_line, 0 };
        pthread_t odd_thread;
        pthread_t even_thread;

        if (start_thread(&odd_thread, &odd_args) == -1) {
            result = EXIT_FAILURE;
            break;
        }

        int even_started = 0;
        if (even_length != -1) {
            if (start_thread(&even_thread, &even_args) == -1) {
                wait_thread(odd_thread);
                result = EXIT_FAILURE;
                break;
            }
            even_started = 1;
        }

        if (wait_thread(odd_thread) == -1) {
            result = EXIT_FAILURE;
        }
        if (even_started && wait_thread(even_thread) == -1) {
            result = EXIT_FAILURE;
        }
    }
}

```

```

        if (odd_args.error || (even_started && even_args.error)) {
            fprintf(stderr, "Ошибка записи в выходной файл.\n");
            result = EXIT_FAILURE;
        }
        if (result != EXIT_SUCCESS) {
            break;
        }
    }

    if (ferror(input)) {
        perror("getline");
        result = EXIT_FAILURE;
    }

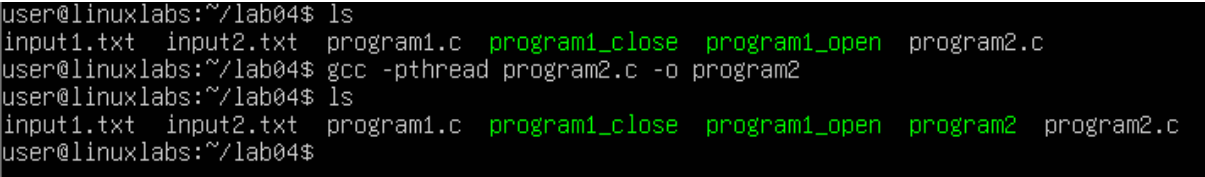
    free(odd_line);
    free(even_line);
    if (fclose(input) == EOF) {
        perror("fclose input");
        result = EXIT_FAILURE;
    }
    if (fclose(odd) == EOF) {
        perror("fclose odd");
        result = EXIT_FAILURE;
    }
    if (fclose(even) == EOF) {
        perror("fclose even");
        result = EXIT_FAILURE;
    }

    return result;
}

```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММЫ

3.1 Компиляция



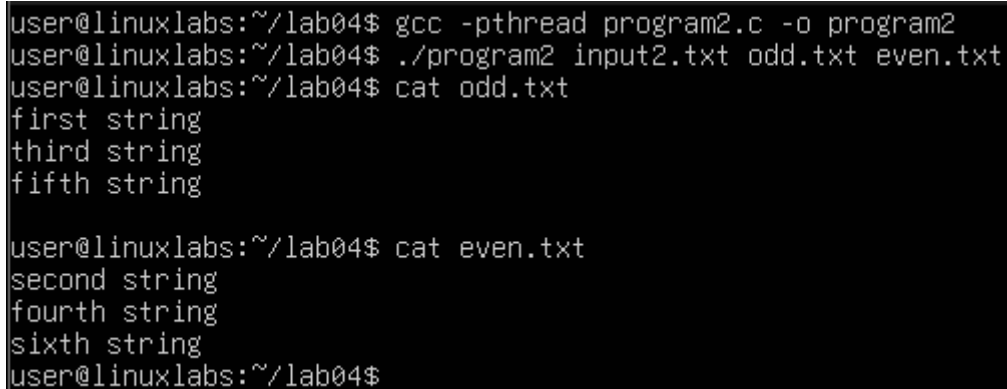
```

user@linuxlabs:~/lab04$ ls
input1.txt  input2.txt  program1.c  program1_close  program1_open  program2.c
user@linuxlabs:~/lab04$ gcc -pthread program2.c -o program2
user@linuxlabs:~/lab04$ ls
input1.txt  input2.txt  program1.c  program1_close  program1_open  program2  program2.c
user@linuxlabs:~/lab04$

```

Рисунок 2 - Компиляция программы

3.2 Запуск и проверка файлов



```
user@linuxlabs:~/lab04$ gcc -pthread program2.c -o program2
user@linuxlabs:~/lab04$ ./program2 input2.txt odd.txt even.txt
user@linuxlabs:~/lab04$ cat odd.txt
first string
third string
fifth string

user@linuxlabs:~/lab04$ cat even.txt
second string
fourth string
sixth string
user@linuxlabs:~/lab04$
```

Рисунок 3 - Результат разделения строк входного файла

4 АНАЛИЗ РЕЗУЛЬТАТОВ

На каждой итерации программа считывает не более двух строк. Первый поток получает строку с нечётным номером и записывает её в odd.txt, второй поток получает следующую, чётную строку и записывает её в even.txt. Потоки используют разные выходные файлы, поэтому дополнительная взаимная блокировка при записи не требуется.

Основной поток вызывает pthread_join для каждого созданного потока до чтения следующей пары строк. Благодаря этому буферы со строками не изменяются, пока дочерние потоки ещё используют их. Если входной файл содержит нечётное количество строк, в последней итерации создаётся только поток для нечётной строки.

ВЫВОД

В ходе лабораторной работы изучены функции pthread_create и pthread_join. Разработана программа, которая двумя потоками распределяет нечётные и чётные строки входного файла по разным выходным файлам.